

Лабораторна робота 5

Тема: апроксимація функцій багатошаровим персептроном (середовище Python)

Цей PDF містить: мету й завдання, таблицю варіантів, покрокову інструкцію з прикладом, типові проблеми, повний текст requirements.txt та повний код lab5_mlp_approximation.py для копіювання.

Мета

Отримати практичні навички підготовки даних у форматах CSV та з роздільниками, створення та навчання багатошарової нейронної мережі для наближення $f(t)$ на $[0; 10]$, збереження моделі та аналізу результатів у Python.

Завдання

1. Створити файл CSV із t та $f(t)$ на $[0; 10]$.
2. Прочитати CSV у Python (аналог csvread).
3. Записати та зчитати дані з роздільником (аналог dlmwrite / dlmread).
4. Побудувати feed-forward мережу для апроксимації $f(t)$ з табл. 1.
5. Зберегти модель у файлі.
6. Побудувати графік бажаних і передбачених значень.
7. Проаналізувати результати та зробити висновки.

Таблиця 1 — $f(t)$, аргумент t на $[0; 10]$

№	$f(t)$	№	$f(t)$
1	$\cos^2(t) \cdot \sin(2t)$	11	$\ln(2t) \cdot \sin(t^{1.5})$
2	$t^2 \cdot \sin(t)$	12	$\sin(t^2 - 8t)$
3	$\sin(0.5t^2)$	13	$0.01t^2 \cdot \sin(t)$
4	$\sin(t^2 - 10t)$	14	$\sin(t^2 - 12t)$
5	$t \cdot \sin(t)$	15	$t \cdot \sin(2t)$
6	$\sin(t^2 - 5t)$	16	$\sin(t^2 - 6t)$
7	$\cos(1.4t)$	17	$t \cdot \sin(3t)$
8	$\sin(t^2 - 8t)$	18	$\sin(t^2 - 4t)$
9	$\ln(t) \cdot \sin(t)^*$	19	$t \cdot \sin(4t)$
10	$\sin(t^2 - 12t)$	20	$(t^2 - 10t) \cdot \sin(t^2 - 10t)$

* Для $\ln(t)$ при $t = 0$ у коді є захист від $\log(0)$.

Покрокова інструкція (крок → одразу дія або команда)

Крок 1.

Дія: скопіюйте теку практичної або створіть нову теку та перенесіть туди файли requirements.txt і lab5_mlp_approximation.py з розділів цієї методички нижче. Усі наступні команди — з цієї теки.

Крок 2.

Дія: встановіть Python 3.10+ з python.org, увімкніть «Add python.exe to PATH». Якщо команда python лише друкує «Python» — це Store; тоді кроки 4–5 робіть через lab.bat / run_lab5.cmd.

Команда перевірки (PowerShell):

```
python --version
```

Крок 3.

Дія: у своїй теці створіть два файли з іменами requirements.txt та lab5_mlp_approximation.py, вставте з PDF повний текст з відповідних розділів нижче, збережіть у кодуванні UTF-8.

Крок 4.

Дія: перший запуск — установка бібліотек і виконання лаби. У Windows можна подвійний клік по run_lab5.cmd у теці з файлами. Або в PowerShell (шлях cd — ваш, де лежать файли лаби):

```
cd $env:USERPROFILE\OneDrive\Documents\lab5_mlp
.\run_lab5.cmd
```

Якщо рір уже виконували — короткий повторний запуск:

```
.\lab.bat
```

Крок 5.

Дія: підставте свій номер варіанту з табл. 1 (--variant 1...20) і прізвище латиницею (--network-name). Одразу повна команда (приклад — замініть число та ім'я):

```
.\lab.bat --variant 7 --network-name Ivanenko
```

Крок 6.

Дія: перегляньте підтеку output (CSV, TXT, .joblib, PNG) та вікно програми з графіком і метриками. У звіт додайте скріншоти, код і висновки за п. 7 методички.

Приклад одним блоком (скопювати в PowerShell)

```
cd $env:USERPROFILE\OneDrive\Documents\lab5_mlp
.\lab.bat --variant 7 --network-name Ivanenko
```

Очікувано: файли на кшталт Ivanenko_data.csv, Ivanenko_plot.png у output. Замініть шлях cd, 7 та Ivanenko на свої значення.

Типові проблеми та рішення

Після команди python у терміналі з'являється лише слово «Python» і помилка.

Це не інтерпретатор, а псевдонім Microsoft Store. Рішення: запускайте .\lab.bat або .\run_lab5.cmd; або вимкніть псевдоніми python.exe / python3.exe (Параметри Windows → Програми → Псевдоніми виконання програм); або викличте Python повним шляхом, наприклад: %LOCALAPPDATA%\Programs\Python\Python312\python.exe

Помилка pip (Permission denied, SSLKEYLOGFILE).

У PowerShell перед установкою виконайте: Remove-Item Env:SSLKEYLOGFILE -ErrorAction SilentlyContinue або закрийте програми, що підміняють змінну SSLKEYLOGFILE.

Не знайдено Python.

Встановіть Python з python.org або виконайте `winget install Python.Python.3.12`. Переконайтесь, що існує файл `Python312\python.exe` у теці `Local\Programs\Python`.

PDF методички немає або треба оновити.

У теці з матеріалами виконайте подвійний клік `pdf.bat` або вручну: `pip install -r requirements-doc.txt`, потім `python generate_student_manual_pdf.py` (через той самий `python.exe`, що й для лаби).

OneDrive показує файли «лише в хмарі».

Дочекайтесь синхронізації або позначте теку «Завжди зберігати на цьому пристрої».

Помилка «не знайдено шрифт» при генерації PDF на іншому ПК.

На Windows має бути `arial.ttf` у `C:\Windows\Fonts`. Інакше змініть шлях до шрифту в `generate_student_manual_pdf.py`.

Відповідність MatLab і Python (коротко)

MatLab	Python
<code>csvread</code>	<code>pandas.read_csv(...).to_numpy()</code>
<code>csvwrite</code>	<code>DataFrame.to_csv(...)</code>
<code>dlmwrite</code> / <code>dlmread</code>	<code>numpy.savetxt</code> / <code>numpy.loadtxt</code>
<code>newff</code> , <code>train</code> , <code>sim</code>	<code>sklearn MLPRegressor</code> (або <code>Keras</code>), <code>fit</code> , <code>predict</code>
<code>save net</code>	<code>joblib.dump</code>

Короткі теоретичні відомості

CSV — обмін табличними даними між Excel і Python (`pandas`). Текстові файли з роздільником — `numpy.savetxt` / `loadtxt` (аналог `dlm`). Багатошаровий перцептрон у прикладі реалізовано через Pipeline з `StandardScaler` та `MLPRegressor` (`logistic`, `lbfgs`).

Файл `requirements.txt` — скопіюйте повністю

Збережіть як `requirements.txt` у теці з лабораторною.

```
numpy>=1.24
pandas>=2.0
matplotlib>=3.7
scikit-learn>=1.3
joblib>=1.3
```

Файл `lab5_mlp_approximation.py` — скопіюйте повністю

Збережіть під іменем `lab5_mlp_approximation.py`. Параметри `--variant` та `--network-name` задаються при запуску (див. приклад вище); у самому файлі їх змінювати не обов'язково.

```
"""
```

Лабораторна робота 5 — переробка методички (MatLab) на Python.

Відповідність пунктів методички див. у попередніх версіях звіту; тут реалізація в Python.

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import sys
```

```
from pathlib import Path
```

```

if hasattr(sys.stdout, "reconfigure"):
    try:
        sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    except (AttributeError, OSError, ValueError):
        pass

try:
    import joblib
    import matplotlib.pyplot as plt
    import numpy as np
    import pandas as pd
    from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
    from sklearn.neural_network import MLPRegressor
    from sklearn.pipeline import Pipeline
    from sklearn.preprocessing import StandardScaler
except ImportError as e:
    print("Не вистачає бібліотек. У терміналі Cursor виконай у цій теці:", file=sys.stderr)
    print(" python -m pip install -r requirements.txt", file=sys.stderr)
    print("Деталі:", e, file=sys.stderr)
    sys.exit(1)

```

```

def variant_function(t: np.ndarray, variant: int) -> np.ndarray:
    """Нелінійні функції з табл. 1. Інтервал завдання: [0, 10]."""
    t = np.asarray(t, dtype=float)
    eps = 1e-9

```

```

    if variant == 1:
        return (np.cos(t) ** 2) * np.sin(2 * t)
    if variant == 2:
        return (t**2) * np.sin(t)
    if variant == 3:
        return np.sin(0.5 * (t**2))
    if variant == 4:
        return np.sin(t**2 - 10 * t)
    if variant == 5:
        return t * np.sin(t)
    if variant == 6:
        return np.sin(t**2 - 5 * t)
    if variant == 7:
        return np.cos(1.4 * t)
    if variant == 8:
        return np.sin(t**2 - 8 * t)
    if variant == 9:
        return np.log(np.maximum(t, eps)) * np.sin(t)
    if variant == 10:
        return np.sin(t**2 - 12 * t)
    if variant == 11:
        return np.log(np.maximum(2 * t, eps)) * np.sin(t**1.5)
    if variant == 12:
        return np.sin(t**2 - 8 * t)
    if variant == 13:
        return 0.01 * (t**2) * np.sin(t)
    if variant == 14:
        return np.sin(t**2 - 12 * t)
    if variant == 15:
        return t * np.sin(2 * t)
    if variant == 16:
        return np.sin(t**2 - 6 * t)
    if variant == 17:
        return t * np.sin(3 * t)
    if variant == 18:
        return np.sin(t**2 - 4 * t)
    if variant == 19:
        return t * np.sin(4 * t)
    if variant == 20:
        return (t**2 - 10 * t) * np.sin(t**2 - 10 * t)

```

```
raise ValueError(f"Невідомий варіант: {variant} (очікується 1–20)")
```

```
def make_dataset(t_min: float, t_max: float, step: float, variant: int) -> tuple[np.ndarray, np.ndarray]:
```

```
    t = np.arange(t_min, t_max + step * 0.5, step, dtype=float)
    y = variant_function(t, variant)
    return t, y
```

```
def _configure_plot_axes(
```

```
    ax,
    t_train: np.ndarray,
    y_train: np.ndarray,
    t_test: np.ndarray,
    y_true: np.ndarray,
    y_pred: np.ndarray,
    variant: int,
```

```
) -> None:
```

```
    ax.plot(t_train, y_train, ".", markersize=4, label="Навчальні точки f(t)")
    ax.plot(t_test.ravel(), y_true, "-", linewidth=1.5, label="f(t) (еталон на тесті)")
    ax.plot(t_test.ravel(), y_pred, "r+", markersize=4, label="Вихід мережі (sim)")
    ax.set_xlabel("t")
    ax.set_ylabel("f(t)")
    ax.set_title(f"ЛП5 (Python): MLP, варіант {variant}")
    ax.legend()
    ax.grid(True, alpha=0.3)
```

```
def _show_results_window(
```

```
    lines: list[str],
    t_train: np.ndarray,
    y_train: np.ndarray,
    t_test: np.ndarray,
    y_true: np.ndarray,
    y_pred: np.ndarray,
    variant: int,
```

```
) -> None:
```

```
    """Зліва — текст як у консолі; справа — графік завдання (без окремої кнопки для PNG)."""
```

```
    try:
```

```
        import tkinter as tk
        from tkinter import scrolledtext
```

```
        from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
        from matplotlib.figure import Figure
```

```
    except ImportError:
```

```
        for ln in lines:
            print(ln)
        return
```

```
    root = tk.Tk()
    root.title("ЛП5 — результат")
    root.geometry("1150x580")
    root.minsize(860, 420)
```

```
    paned = tk.PanedWindow(root, orient=tk.HORIZONTAL, sashrelief=tk.RAISED, sashwidth=5)
    paned.pack(fill=tk.BOTH, expand=True, padx=4, pady=4)
```

```
    left = tk.Frame(paned)
    st = scrolledtext.ScrolledText(left, wrap="word", font=("Segoe UI", 10))
    st.pack(fill=tk.BOTH, expand=True, padx=4, pady=4)
    st.insert("1.0", "\n".join(lines))
    st.configure(state="disabled")
    paned.add(left, minsize=340)
```

```
    right = tk.Frame(paned)
    fig = Figure(figsize=(6.5, 5.0), dpi=100)
```

```

ax = fig.add_subplot(111)
_configure_plot_axes(ax, t_train, y_train, t_test, y_true, y_pred, variant)
fig.tight_layout()
canvas = FigureCanvasTkAgg(fig, master=root)
canvas.draw()
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True, padx=4, pady=4)
paned.add(right, minsize=500)

tk.Button(root, text="Закрити", command=root.destroy).pack(pady=6)

root.mainloop()

def main() -> None:
    _gui_default = sys.platform == "win32"
    p = argparse.ArgumentParser(description="ЛР5: MLP-апроксимація f(t) на [0,10]")
    p.add_argument("--variant", type=int, default=1, choices=range(1, 21), help="Номер варіанту з табл. 1")
    p.add_argument("--step", type=float, default=0.05, help="Крок сітки по t для навчання")
    p.add_argument("--out-dir", type=Path, default=Path(__file__).resolve().parent / "output")
    p.add_argument(
        "--network-name",
        type=str,
        default="student_network",
        help="База імені файлів (у звіті — латиницею прізвище)",
    )
    p.add_argument(
        "--gui",
        action=argparse.BooleanOptionalAction,
        default=_gui_default,
        help="Вікно: текст зліва, графік справа (за замовч. на Windows). / --no-gui — консоль.",
    )
    args = p.parse_args()
    out_dir = args.out_dir
    out_dir.mkdir(parents=True, exist_ok=True)

    base = args.network_name
    csv_path = out_dir / f"{base}_data.csv"
    dlm_path = out_dir / f"{base}_data.txt"
    model_path = out_dir / f"{base}.joblib"
    plot_path = out_dir / f"{base}_plot.png"

    t_train, y_train = make_dataset(0.0, 10.0, args.step, args.variant)
    df = pd.DataFrame({"t": t_train, "f": y_train})
    df.to_csv(csv_path, index=False)

    M_csv = pd.read_csv(csv_path).to_numpy()
    x1 = M_csv[:, 0].reshape(-1, 1)
    y1 = M_csv[:, 1].ravel()

    np.savetxt(dlm_path, M_csv, delimiter="\t", fmt="%.8e")
    M_dlm = np.loadtxt(dlm_path, delimiter="\t")
    if not np.allclose(M_csv, M_dlm):
        raise RuntimeError("Розбіжність після dlm-циклу: перевірте формат файлу.")

    net = Pipeline(
        [
            ("scaler", StandardScaler()),
            (
                "mlp",
                MLPRegressor(
                    hidden_layer_sizes=(5, 3),
                    activation="logistic",
                    solver="lbfgs",
                    alpha=1e-6,
                    max_iter=8000,
                    random_state=42,
                )
            )
        ]
    )

```

```

        tol=1e-6,
    ),
),
]
)
net.fit(x1, y1)

joblib.dump(net, model_path)

t_test = np.arange(0.025, 10.0, 0.1, dtype=float).reshape(-1, 1)
y_true = variant_function(t_test.ravel(), args.variant)
y_pred = net.predict(t_test)

mse = mean_squared_error(y_true, y_pred)
mae = mean_absolute_error(y_true, y_pred)
r2 = r2_score(y_true, y_pred)

lines: list[str] = [
    "=====",
    "ЛР5 — підсумок по пунктах методички",
    "=====",
    f"Варіант з табл. 1: {args.variant}",
    f"База імені файлів (мережа): {base}",
    f"Крок сітки t для навчання: {args.step}",
    """
    ,
    "————— Завдання 1 —————",
    "За методичкою: підготувати файл .csv з даними f(t) на [0; 10].",
    f"Тут: CSV згенеровано автоматично (стовпці t та f).",
    f"Файл: {csv_path}",
    """
    ,
    "————— Завдання 2 —————",
    "За методичкою: прочитати csv у програмі (у MatLab — csvread).",
    "Тут: файл прочитано через pandas.read_csv → масив для навчання.",
    f"Перевірка: ті самі дані використано для стовпців t та f.",
    """
    ,
    "————— Завдання 3 —————",
    "За методичкою: записати й зчитати файл з роздільником (dlmwrite / dlmread).",
    "Тут: numpy.savetxt / loadtxt з табуляцією.",
    f"Файл: {dlm_path}",
    "Перевірка: масив після читання збігається з CSV.",
    """
    ,
    "————— Завдання 4 —————",
    "За методичкою: мережа feed-forward з навчанням backprop для наближення f(t).",
    "Тут: sklearn — Pipeline(StandardScaler + MLPRegressor), шари (5, 3), активація
logistic, solver lbfgs.",
    "Навчання виконано на усіх точках навчальної сітки.",
    """
    ,
    "————— Завдання 5 —————",
    "За методичкою: зберегти навчену мережу у файлі.",
    f"Тут: joblib.dump → файл моделі.",
    f"Файл: {model_path}",
    """
    ,
    "————— Завдання 6 —————",
    "За методичкою: на одному графіку — бажані та обчислені значення на виході мережі.",
    f"Тут: графік збережено у PNG; той самий графік показано справа у цьому вікні.",
    f"Файл: {plot_path}",
    """
    ,
    "Метрики на тестовій сітці (для аналізу у звіті, п. 7):",
    f" MSE = {mse:.6e}",
    f" MAE = {mae:.6e}",
    f" R^2 = {r2:.6f}",
    """
    ,
    "===== Завдання 7 і текст «висновки» у звіті =====",
    "За методичкою п. 7: проаналізувати результати і зробити висновки — це ВИ пишите",
    "у звіті самостійно (проза): що показує графік, наскільки малі MSE/MAE,",
    "чи задовольняє наближення, що можна змінити (крок, архітектура, епохи тощо).",
    """
    ,

```

```

        "Нижче — НЕ готовий звіт і не з методички дослівно: це лише автоматична",
        "однорядкова підказка-чернетка від програми (можна ігнорувати або перефразувати у
звіті):",
        """
        " » двошаровий персептрон (5 і 3 нейрони, logistic) навчено наближати  $f(t)$  на [0; 10].
        ",
        "    За графіком і метриками оцініть якість самостійно.",
    ]

    if args.gui:
        _show_results_window(lines, t_train, y_train, t_test, y_true, y_pred, args.variant)
    else:
        for ln in lines:
            print(ln)

if __name__ == "__main__":
    main()

```

Зміст звіту студента

1. Титульна сторінка.
2. Тема та мета.
3. Завдання.
4. Опис виконання з рисунками (термінал, код, графік з output).
5. Текст програми (можна посилатися на файл з цієї методички).
6. Висновки.